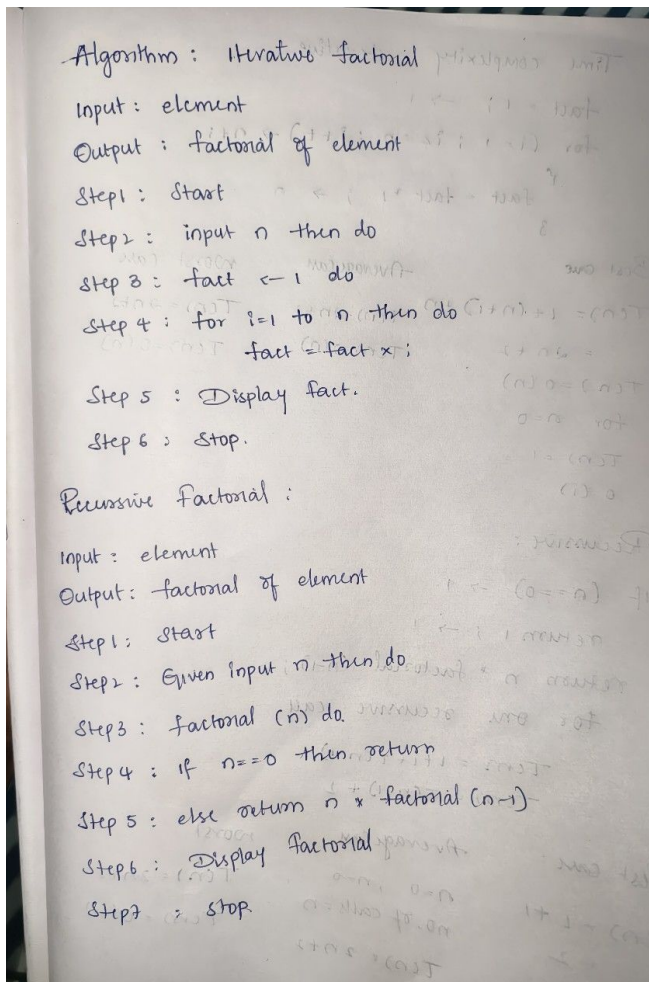


Practical: 4

Implementation and Time analysis of factorial program using iterative and recursive method

Aim: To Implement and Time analysis of factorial program using iterative and recursive method

Algorithm:



Program:

Iterative Code

```
#include<iostream>
using namespace std;

int factorial(int n){
    int fact = 1;

    for(int i = 1; i <= n; i++){
        fact = fact * i;
    }

    return fact;
}

int main(){
    int n;

    cout<<"enter the number ";
    cin>>n;

    cout<<"factorial of the number is: "<<factorial(n);

    return 0;
}
```

Recursive Code

```
#include<iostream>
using namespace std;

int factorial(int n){
    if(n==0){
        return 1;
    }
    else{
        return n*factorial(n-1);
    }
}

int main(){
    int n;
    cout<<"enter the number ";
    cin>>n;
    cout<<"factorial of the number is: "<<factorial(n);
    return 0;
}
```

Output:

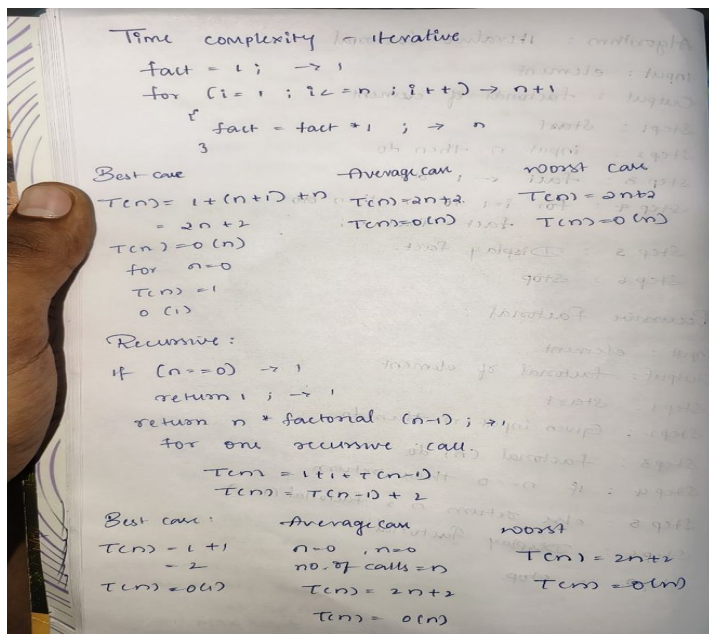
Iterative Method

```
Enter the number 5
Factorial of the number is: 120
-----
Process exited after 3.639 seconds with return value 0
Press any key to continue . . . |
```

Recursive Method

```
Enter the number 6
Recursive Method Factorial of the number is: 720
-----
Process exited after 1.667 seconds with return value 0
Press any key to continue . . . |
```

Time Complexity:



Conclusion:

Hence, the Max-Heap Sort algorithm was successfully implemented and executed. It was observed that Heap Sort gives a time complexity of $O(n \log n)$ in the best, average, and worst cases. It is an efficient sorting technique and works well for sorting large sets of data.